

20. SHAP LIME XAI Deployment

Overview

Explainability methods attribute predictions to input features. SHAP uses Shapley values from cooperative game theory; LIME fits local interpretable surrogates. The module closes with serving considerations: logging explanations alongside predictions in a small Flask deployment.

Lab: Lab 20.

Prior modules: Module 11.

Contents

1	Why Explainability?	2
2	SHAP Values	2
3	Shapley Kernel	2
4	LIME	2
5	Model-Specific Accelerations	2
6	Deployment Integration	2
7	Limitations	3
8	Lab	3
9	Global vs. Local Explanations	3
9.1	Regulatory Context	3
10	Extended Intuition	4
11	Numerical Walkthrough	4
12	PyTorch Implementation Notes	4
13	Local GPU Constraints	5
14	Debugging Checklist	5
14.1	Explanation Fidelity, Correlation, and Monitoring	5
15	Practice Problems	6

Module track

This module builds on **Module 11**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

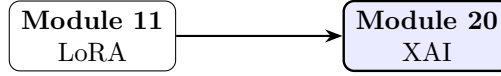


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Why Explainability?

Deployed models from **Module 19** (MLflow tracking) affect users; regulators and stakeholders demand interpretability. Post-hoc methods explain black-box predictions without changing model internals.

2 SHAP Values

SHAP [3] assigns feature attributions ϕ_i satisfying efficiency:

$$\sum_{i=1}^M \phi_i = f(\mathbf{x}) - \mathbb{E}[f(\mathbf{X})], \quad (1)$$

derived from Shapley values in cooperative game theory.

Note. SHAP provides unified framework: LIME (section 4), DeepLIFT, and TreeSHAP are approximations under specific assumptions.

3 Shapley Kernel

Weighted linear regression on coalition indicators $\mathbf{z} \in \{0, 1\}^M$:

$$\pi_{\mathbf{x}}(\mathbf{z}) = \frac{(M-1)!}{\binom{M}{|\mathbf{z}|} |\mathbf{z}|! (M-|\mathbf{z}|)!}. \quad (2)$$

Kernel SHAP estimates (1) by sampling coalitions.

4 LIME

LIME [5] fits local interpretable model g near $\mathbf{x}$:

$$\xi(\mathbf{x}) = \arg \min_{g \in \mathcal{G}} \mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g), \quad (3)$$

where $\mathcal{L}$ measures fidelity of g to black-box f on perturbed samples weighted by $\pi_{\mathbf{x}}$.

5 Model-Specific Accelerations

TreeSHAP: exact Shapley for tree ensembles in polynomial time. Gradient SHAP: integrates gradients for neural nets fine-tuned via LoRA in **Module 11** (LoRA definition).

6 Deployment Integration

Serving pipeline returns $(\hat{y}, \phi)$ for audit logs. Monitor attribution drift when input distribution shifts (see **Module 19** (MLflow tracking)).

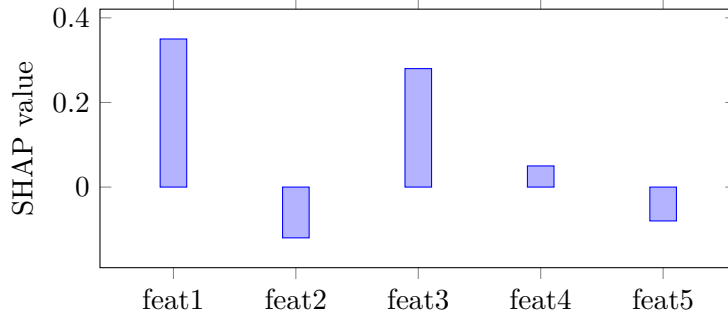
Table 1: XAI method selection.

Model	Method	Scope
Any classifier	LIME	Local
Tree ensembles	TreeSHAP	Global + local
Deep networks	Gradient SHAP	Local
LLM	Attention rollout	Token-level

7 Limitations

Explanations are not causal; correlated features split attribution arbitrarily. Adversarial perturbations can fool LIME/SHAP without changing prediction.

Consistency: if $f(\mathbf{x}) = f(\mathbf{x}')$, SHAP assigns same ϕ_i when contributions identical. (4)

Figure 2: Waterfall-style SHAP contributions to $f(\mathbf{x})$.

8 Lab

Lab 20 computes SHAP and LIME on a deployed classifier; compare attributions from (1) and (3). Integrate explanation logging into the serving stack from **Lab 19**.

9 Global vs. Local Explanations

SHAP summary plots aggregate $|\phi_i|$ across dataset; partial dependence shows marginal effect:

$$PD_j(x_j) = \mathbb{E}_{\mathbf{x}_{\setminus j}}[f(x_j, \mathbf{x}_{\setminus j})]. \quad (5)$$

Counterfactual explanations: minimal change δ s.t. $f(\mathbf{x} + \delta) \neq f(\mathbf{x})$.

9.1 Regulatory Context

EU AI Act high-risk systems may require documentation of explanation methodology and failure modes.

10 Extended Intuition

SHAP attributes a prediction to input features using Shapley values from cooperative game theory: fair distribution of payout across players (features). For tree models, TreeSHAP is fast and exact; for neural nets, KernelSHAP or DeepSHAP approximate with background samples. LIME fits a local linear surrogate around one prediction by perturbing inputs and weighting samples by proximity.

Global importance (mean $|\text{SHAP}|$) differs from local explanations for a single row; both answer different stakeholder questions. Explanation methods can disagree, especially under correlated features; never treat attributions as causal.

Deployment adds latency and storage: log top- k features per request, cache explanations for batch offline review, or compute async. Fine-tuned models from **Module 11** and training tracked in **Module 19** should expose the same preprocessing in serving and explanation paths.

Regulatory contexts may require human-readable reason codes derived from ranked attributions.

11 Numerical Walkthrough

Binary classifier on 5 tabular features; one instance $x = [1.2, 0.0, 3.4, -0.5, 2.1]$, prediction $p = 0.82$.

SHAP values $\phi = [0.15, -0.05, 0.30, 0.02, 0.10]$ sum with base value $\phi_0 = 0.30$ to logit or prob link: $0.30 + 0.15 - 0.05 + 0.30 + 0.02 + 0.10 = 0.82$ (prob space illustration; exact link depends on model output).

LIME: generate 500 perturbed samples around x (Gaussian on numeric features), get predictions p_i , fit weighted linear model with weights $K(x, x_i) = \exp(-\|x - x_i\|^2/\sigma^2)$, $\sigma = 0.5$. Surrogate $R^2 = 0.75$ on neighborhood indicates local linear approximation quality.

Latency budget: TreeSHAP 2 ms, KernelSHAP 200 ms for same model; batch 100 explanations for offline audit vs real-time API.

Flask endpoint returns `{"prediction": 1, "shap": {"f0": 0.15, ...}}` JSON; log alongside input hash for GDPR-aware retention policies.

12 PyTorch Implementation Notes

- `shap.Explainer(model, background_data)` auto-selects algorithm; for sklearn trees use `TreeExplainer`.
- Deep explainer: background tensor shape (50, C, H, W) for images; keep background small on 1650.
- `shap_values = explainer(x)`; force plot with `shap.plots.waterfall` for one row.
- LIME: `lime.lime_tabular.LimeTabularExplainer(training_data, feature_names=..., class_names=...)`.
- Wrap PyTorch model with `predict_fn` that applies softmax and returns numpy for SHAP/LIME.

```
def predict_fn(x):
    model.eval()
    with torch.no_grad():
        t = torch.tensor(x, dtype=torch.float32, device=device)
        return model(t).softmax(1).cpu().numpy()

explainer = shap.KernelExplainer(predict_fn, background_numpy)
sv = explainer.shap_values(instance, nsamples=100)
```

13 Local GPU Constraints

KernelSHAP with 1000 coalitions on large neural nets is slow on 1650; reduce `nsamples` to 50 to 100 for demos. Background dataset 50 to 100 rows balances fidelity vs time; use k-means summary of train set (`shap.kmeans`).

Image DeepSHAP at 224px: batch background 8, expect seconds per explanation; run offline not in request path. LIME on tabular is CPU-cheap; prefer for real-time alongside tree models.

14 Debugging Checklist

- SHAP sum does not match prediction: wrong link function (logit vs prob) or missing base value.
- All zero attributions: model constant on perturbations; check preprocessing mismatch vs training.
- LIME unstable across runs: increase samples or fix random seed on perturbations.
- Explainer OOM on GPU: move background and explain batch to CPU for KernelSHAP.
- Serving mismatch: training scaler not applied in `predict_fn` used by explainer.

14.1 Explanation Fidelity, Correlation, and Monitoring

SHAP values satisfy efficiency: $\sum_i \phi_i + \phi_0 = f(x)$ for the chosen model output link (logit or probability). TreeSHAP exploits tree structure for exact Shapley values in polynomial time; KernelSHAP treats the model as a black box and samples coalitions, trading accuracy for generality. Background dataset defines the reference expectation ϕ_0 ; a background of all zeros on standardized features is wrong if training mean was non-zero. Use `shap.kmeans(train, 50)` to summarize background when raw train has 100k rows; 50 centroids usually capture enough distribution for KernelSHAP.

LIME trains weighted linear surrogate g locally: minimize $\sum_i \pi_x(z_i)(f(z_i) - g(z_i))^2 + \Omega(g)$ where π_x is proximity kernel. High R^2 on perturbed samples means local linearity holds; low R^2 warns the explanation may mislead for that instance. Correlated features (zip code and city) receive split credit that is not uniquely identifiable; report grouped explanations or drop one feature from the set when presenting to stakeholders.

Production patterns: async explanation queue for batch review, sync TreeSHAP for low-latency tree models, precompute SHAP for static batch scoring jobs. Log explanation payloads with model version from MLflow (**Module 19**) so drift investigations tie back to the exact checkpoint. Monitor mean $|\phi_i|$ per feature over sliding windows; sudden spikes in a feature's average attribution often precede data drift alerts. Adversarial inputs can produce misleading attributions; cross-check with holdout performance and LIME on the same row before trusting a single method. For neural models fine-tuned with LoRA (**Module 11**), run explainer on merged weights so `predict_fn` matches serving graph exactly. GDPR-style retention: store hashed inputs and top-3 attributions, not full SHAP vectors, when regulations limit sensitive feature logging.

Interventional vs observational SHAP: interventional breaks correlations by replacing feature values from background independently; observational preserves joint distribution. Pick interventional when stakeholders need "what if we change this one input"; pick observational when features naturally co-occur and you want realistic counterfactuals. Force plots and waterfall charts communicate local attributions to non-technical reviewers better than raw ϕ vectors. Batch explanation jobs should cap `nsamples` per row and shard across workers; log wall time per 1000 rows for capacity planning. SHAP interaction values $\Phi_{i,j}$ capture pairwise feature effects; compute only for top features when $p > 20$ to avoid exponential coalition cost. LIME for text: perturb by deleting word subsets, not Gaussian noise; use `LimeTextExplainer` with same tokenizer as model

input pipeline. Fairness review: compare mean SHAP of sensitive attribute across demographic slices; large disparity triggers model audit even when AUC looks acceptable. Explanation cache key: hash (model_version, preprocessing_hash, input_row); invalidate cache on any scaler or vocab change from **Module 19** deployment.

15 Practice Problems

Problem 1. For 3 features with given SHAP values, verify they sum to prediction minus base rate in logit space.

Problem 2. Compare SHAP vs LIME top-3 features on 10 test rows; measure agreement rate.

Problem 3. Add explanation logging to Flask app from **Lab 19** stack; measure p95 latency overhead.

Problem 4. Identify two correlated features where SHAP splits credit arbitrarily; discuss mitigation (grouped SHAP or drop one).

Complete **Lab 20**; integrates models from **Module 11** and tracking from **Module 19**.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.
- [2] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, et al. A survey of methods for explaining black box models. *ACM Computing Surveys*, 2018.
- [3] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *NeurIPS*, 2017.
- [4] Christoph Molnar. *Interpretable Machine Learning*. Leanpub, 2020.
- [5] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “why should i trust you?” explaining the predictions of any classifier. In *KDD*, 2016.